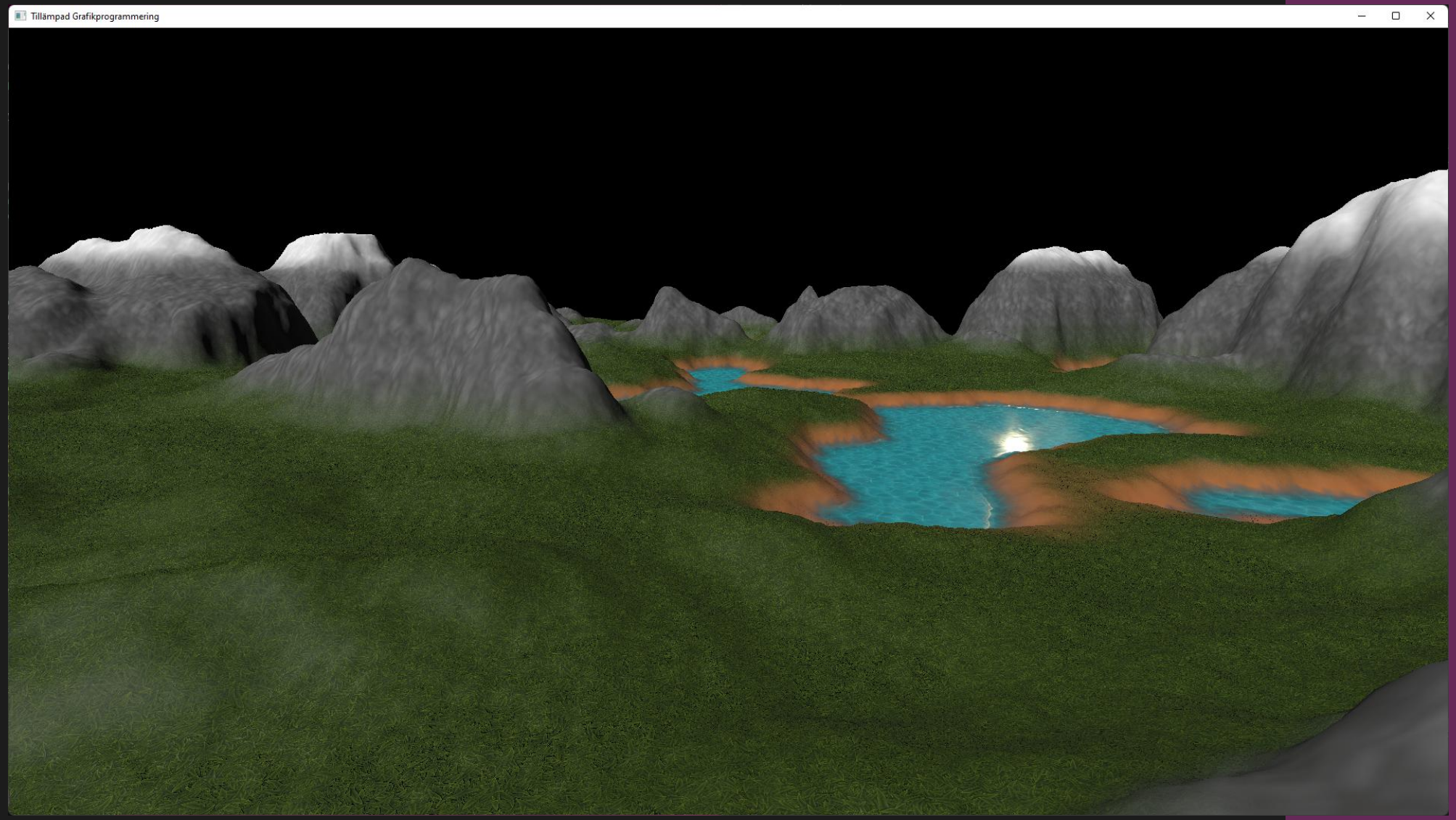
TILLÄMPAD GRAFIKPROGRAMMERING

F?? – HEIGHTFIELDS OCH TERRÄNG

IDAG

- Flera av er har frågat om lite mer kring terräng.
  - SpecielIt hur man genererar det.
- Därför ska i kika lite på detta idag.
  - Dock endast en extraföreläsning. Ingen uppgift!
- Endast tänkt som en teoriföreläsning.
  - Väldigt lite kod, bara små exempel.
- Vi tar lite prog art som exempel på vad som menas...

IDAG



TERRAIN

- Flera av er har kanske kikat på detta innan.
  - Finns flera metoder för att skapa terräng för spel.
- Kan t.ex. skapas via olika 3D program
  - T.ex. Maya, Bryce, Houdini, Substance Designer, etc.
- Kan också skapas proceduellt i kod.
- Vilken metod som är bäst lämpad beror på projektet.

TERRAIN

- Oftast kan program skapa terräng i två olika format.

- Mesh – Detta fungerar precis som andra meshes. Ni läser in och ritar.

- Height Field – Ett fält som håller höjdvärden.
  - Kan sparas i texturform och kallas ofta för Height Map.
    - Dock inte att förväxla för Height Maps för andra ändamål!
  - Värdet i vårt Height Field kontrollerar höjdskillnader i terrängen.

TERRAIN

- I grund och botten är terräng i spel ett eller flera plan.
  - Även om de kommer som Meshes från ett program började de livet som ett plan.
- Ofta stora objekt, bra att dela upp i celler.
  - Lättare att culla bort när det behövs.
- Idag kommer vi dock att bara kika på terräng med en mesh.
  - Vi delar alltså inte upp den i bitar.

TERRAIN

- Eftersom ni redan vet hur man laddar in meshes kommer vi inte att titta på den typen.

- Det blir istället terräng från ett Height Field.
  - Antingen från en Height Map eller egen generation i kod.

- Oavsett metod är ett Height Field ett fält med något skalärvärde per koordinat.

- Behöver inte vara högupplöst. Kommer långt med t.ex. 256 värden per axel.

HEIGHT FIELDS

- I sin simplaste form är ett Height Field i de flesta fallen en 2D array.
  - Sidorna är lika långa, t.ex. 128 x 128
  - Varje koordinat har ett floatvärde.

- Behöver dock inte vara float, kan vara vad som helst.
  - Enkel att göra en Template Class av.
  - Kan t.o.m. specificera dimensionen som template argument.
  - Lättare att minnespacka.
  - Överkurs, sorry x)

```
template<typename T>
class HeightField
{
    std::vector<std::vector<T>> myData{};

    size_t myWidth{}, myHeight{};

public:

    bool Initialize( ... );

    T GetValue(size_t X, size_t Y);
};
```

```
template<typename T, size_t D>
class HeightField
{
    std::array<std::array<T, D>, D> myData{};

public:

    bool Initialize( ... );

    T GetValue(size_t X, size_t Y);
};
```

HEIGHT FIELDS

- Hur får vi då data i vårt Height Field?
- Visst vi kan sitta och stoppa in värden manuellt.
  - Eller i en for-loop!
- Vi kan läsa in data från en textur skapad i något program.
- Eller t.o.m. skapa data proceduellt med noisefunktioner.
- På TGA kommer ni nog mest att använda texturer.

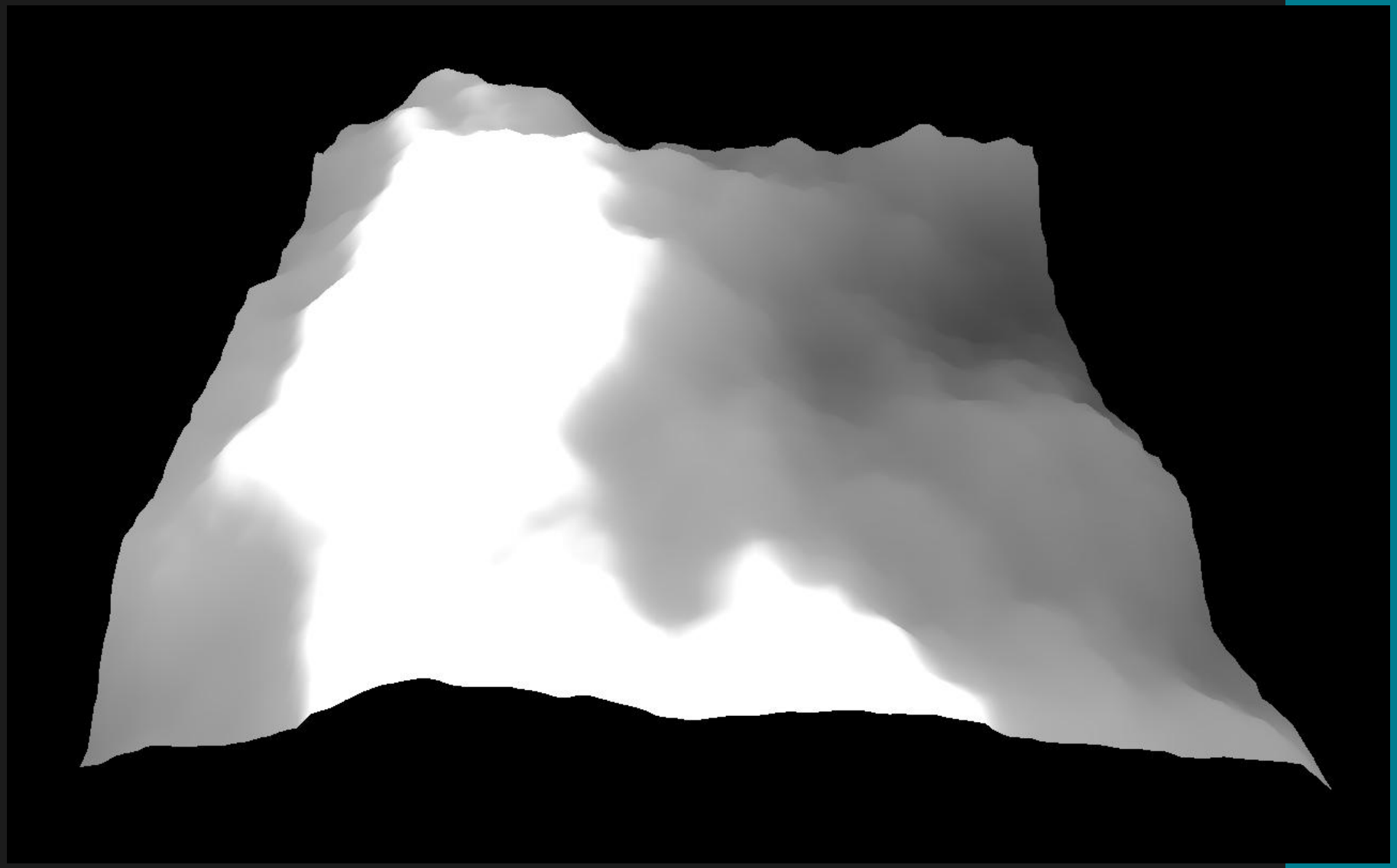
HEIGHT FIELDS

- Vi skall kika lite på texturinladdning av Height Fields om en liten stund...
  - Men först vill jag bara gå igenom ett par noise funktioner!

- Att använda dem är oftast väldigt enkelt.
  - Dubbel for loop, för x och y.
  - Mata in koordinater, få ut värde.
  - Lägg värdet i ert Height Field.

- Mycket experimenterande :).

EXEMPEL HEIGHT FIELD



NOISE

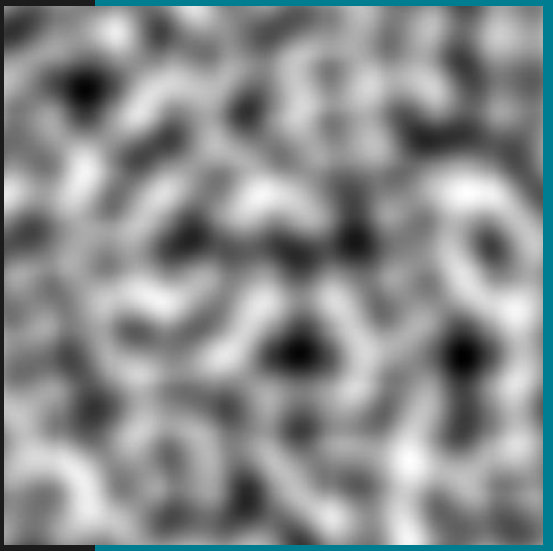
- Noise funktioner används väldigt mycket i olika delar av spelutveckling.
  - Grafik, Programmering, Level Design, mm.

- Vad är en noise funktion?
  - *”En typ av matematisk funktion av reella tal som varierar mellan två maxima i någon domän.”*
  - Nu börjas det igen…

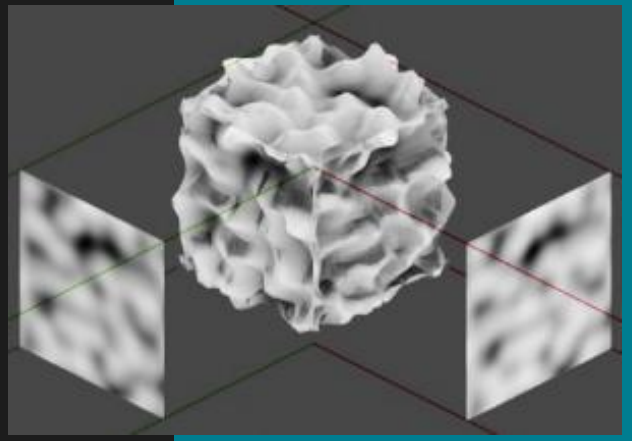
- Vi kan simplifiera beskrivningen för vårt fall
  - En matematisk funktion som producerar ett fält där varje cell har ett värde mellan 0 och 1.

NOISE

- Det första vi tittar på är Perlin Noise.
  - Ken Perlin blev sur på att CGI såg datorgenererat ut.
  - 1983 skapade han Perlin Noise efter han jobbat på Tron filmen.
  - Förmodligen mest använda noise funktionen i 3D grafik.

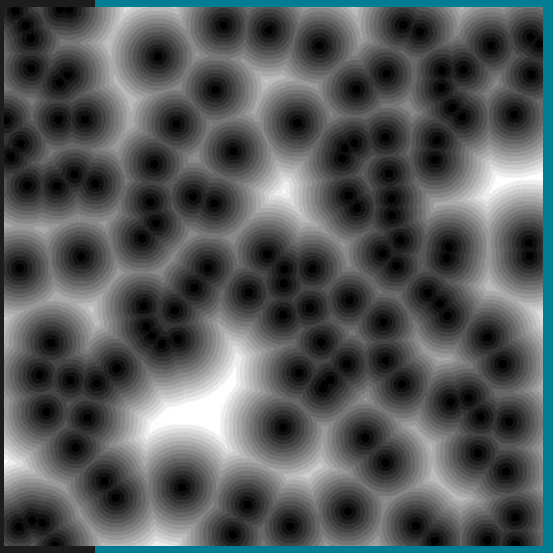


- Lagom dyrt att beräkna i realtid.
  - Många spelmotorer har en färdig 3D textur med flera Perlin snapshots.
    - Lätt att modifiera med olika funktioner, t.ex pow.
  - Finns snabbare versioner med mindre variation
    - t.ex. Simplex Noise.

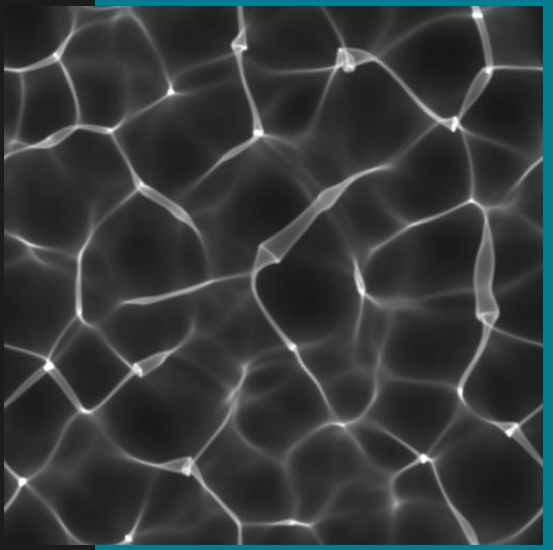


NOISE

- Worley Noise
  - Steven Worley 1996.
  - Skapades för att simulera sten, vatten, celler, mm.

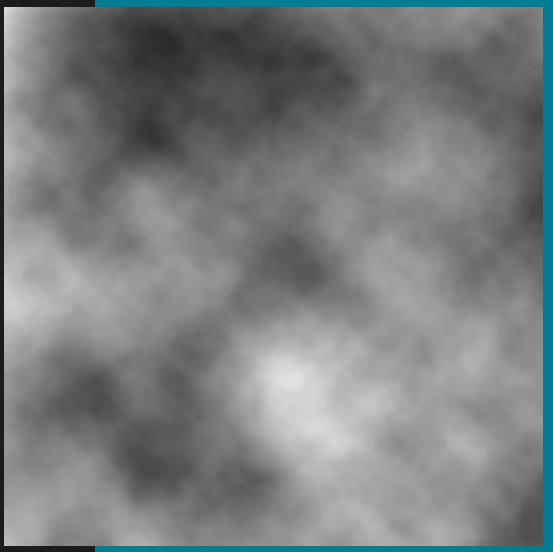


- Exempel där Worley Noise är konfigurerat och multiplicerat för att skapa Caustics.
  - Reflekterade och Refrakterade ljusstrålar genom t.ex. en vätska.
  - Ofta använt för att skapa ljusreflektioner från vatten på t.ex. poolgolv.



NOISE

- Fractal Noise
  - Specifikt Fractal Noise Sum
  - Ofta genererad med flera typer av noise.
  - Just denna är två Perlin Noise skalade och adderade.



- Naturen gillar fraktaler.
  - Ser bra ut i naturen. Kan ibland uppfattas som repeterande mönster i spel.

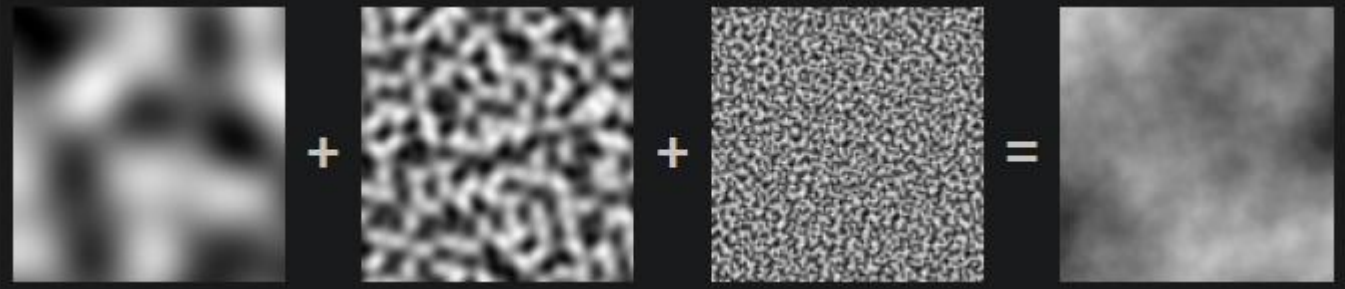


- Väldigt bra för att simulera moln och andra typer av gaser.

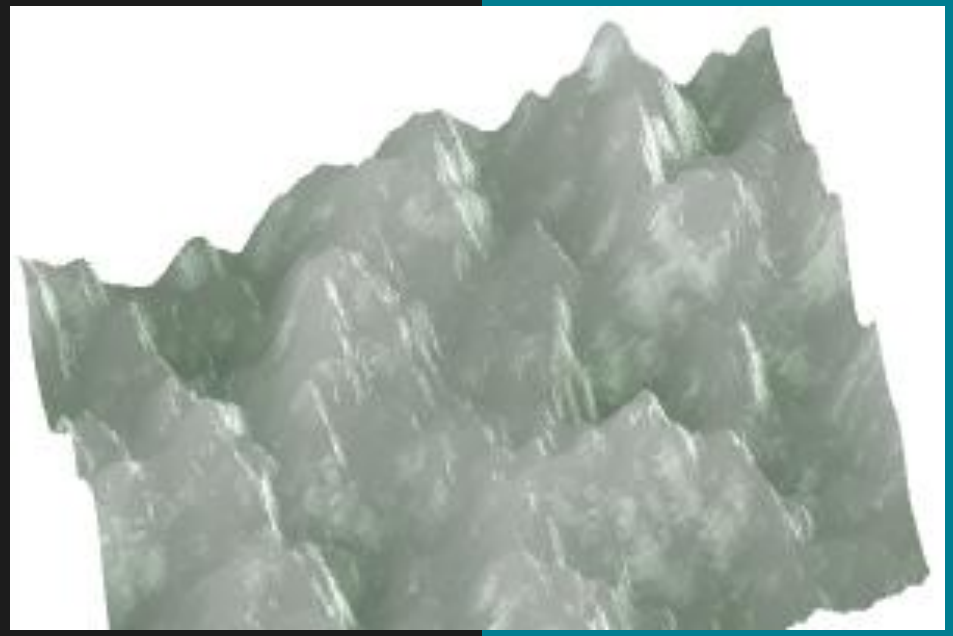
FRACTAL NOISE

- Fractal Noise är ofta väldigt bra på landskap.

- Om vi lägger samman flera olika Perlin Noise steg.
  - Även kallat Octaves, Oktaver.



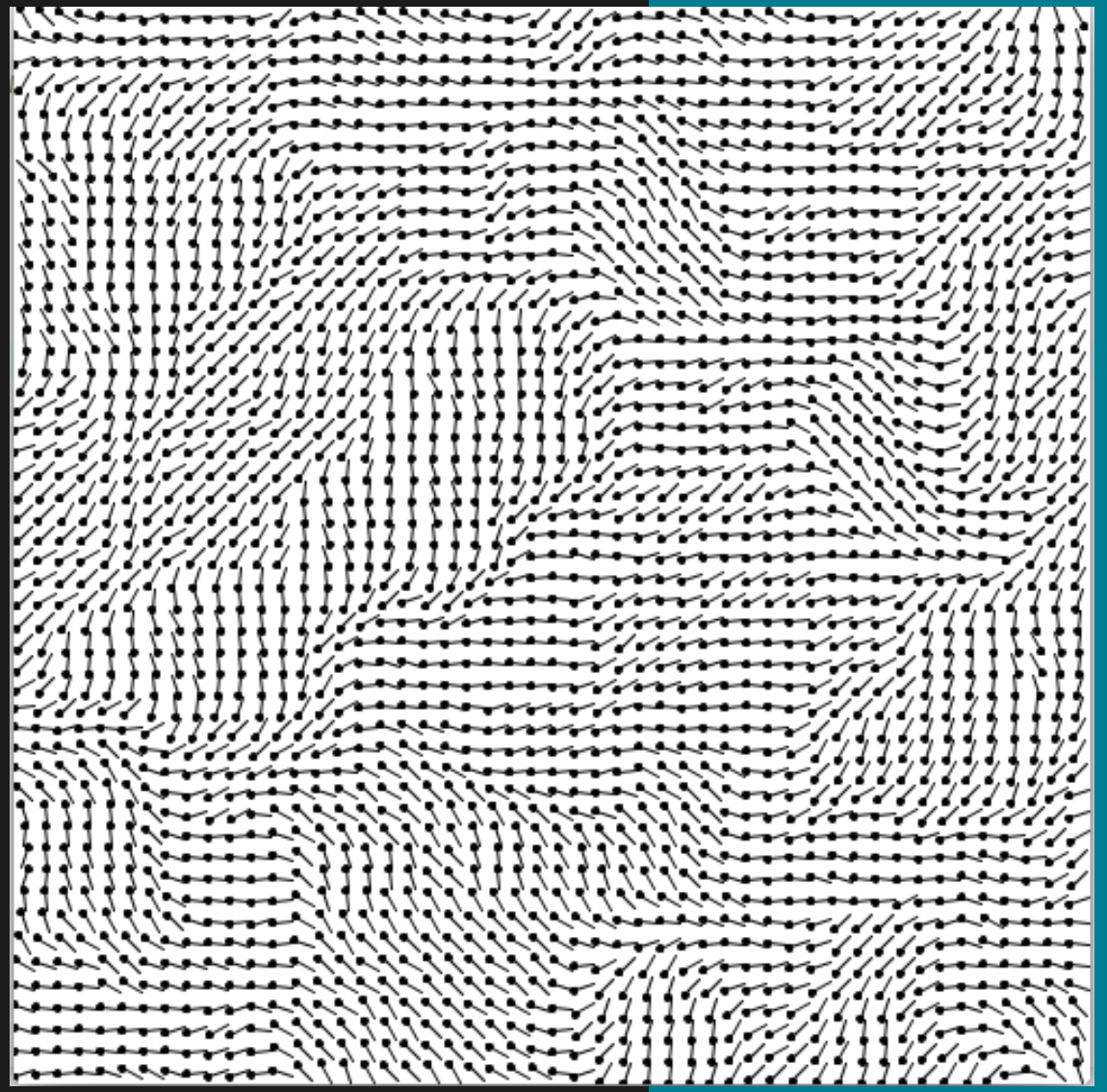
- Jobbar ni med procedural terräng är detta viktigt att tänka på!



NOISE

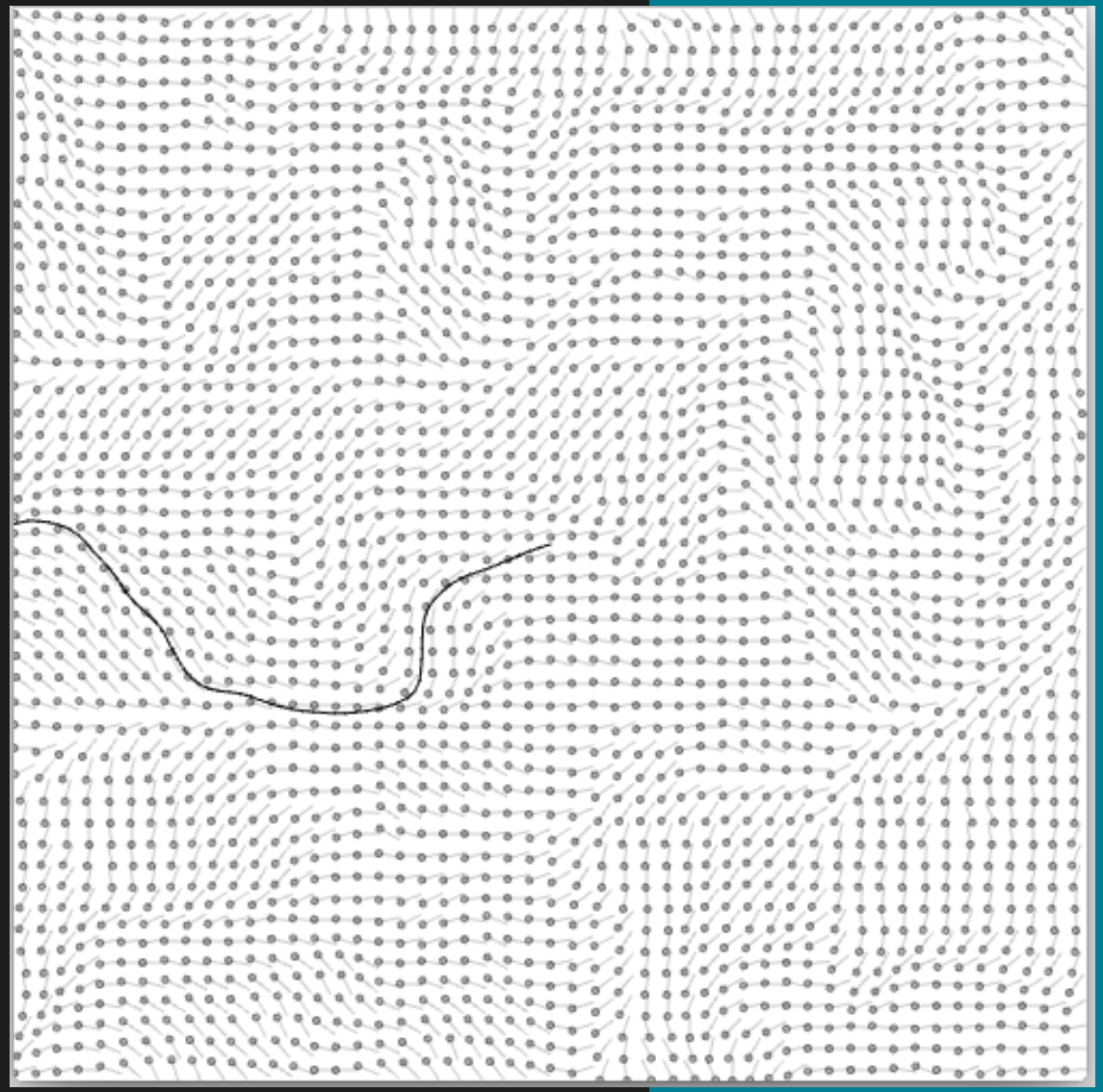
- Flow eller Noise Fields
  - Extramaterial.

- Noise behöver inte ses som en bild.
  - Kan vara andra saker.
  - Tänk om 0-1 är en hel rotation, alltså 0-360.
  - Använd som riktning för att skapa ett Flow Field.
  - Här genererat från Perlin.



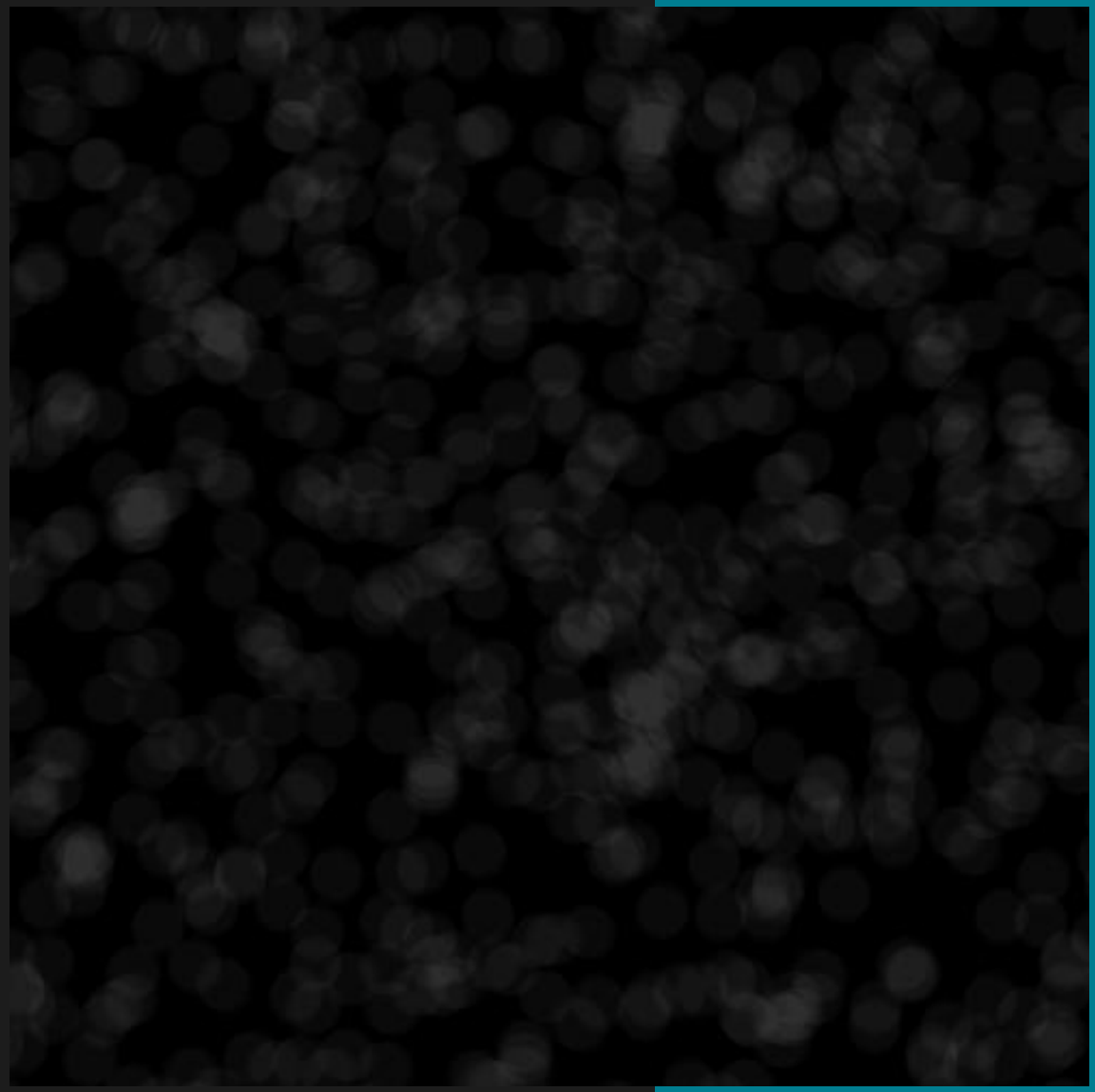
NOISE

- Kan redan användas för olika saker.
  - T.ex. vattenflöde.
  - Men vi kan också rita med det…
- Vi släpper iväg partiklar som ritar på vår textur när de rör sig…
  - Har jitter och annan data för att inte få samma hela tiden.



NOISE

- Kan redan användas för olika saker.
  - T.ex. vattenflöde.
  - Men vi kan också rita med det…
- Vi släpper iväg partiklar som ritar på vår textur när de rör sig…
- Gör vi detta flera gånger får vi organiska och intressanta former.



NOISE

- Finns flera andra, för många att gå in på.
- Hur använder vi t.ex. Perlin Noise eller en Height Map?
- Finns flera metoder.
  - Vi kan t.ex. använda den direkt i Vertex Shadern.
    - Väldigt dyrt, körs varje frame.
    - Svårt att komma åt data på CPUn för kollisioner.
  - Eller så kan vi använda den att skapa en mesh direkt.
    - Då har vi redan meshdata att läsa på CPUn mm.
- Det var Noise. Nu kan vi titta på Height Maps.

TGA

TEXTURDATA

- En Height Map är som sagt bara ett Height Field sparat som en textur.
- Betyder att vi måste läsa in den och konvertera.
- Här är det viktigt att tänka på att vi alltid jobbar med DDS filer.
  - DDS filer är nästan alltid komprimerade.
  - Sparar plats och minne.
  - Måste packas upp för att användas utanför grafikkortet.

TEXTURDATA

- Vi kan redan ladda DDS filer.
  - Albedo, Normaler, Material, etc.
- Dock kan vi inte komma åt datan på ett enkelt sätt.
  - Det var det där med komprimeringen.
  - Vi använder BC7 som inte är SÅ svårt.
    - Dock bra att stödja flera format.
- Lyckligtvis finns det färdig kod att använda.
  - Microsoft har skapat DirectXTex.
  - DirectX Texture Processing Library.

TEXTURDATA

- Finns på PingPong ihop med föreläsningen.
  - Och på github under DirectXTex texture processing library
  - Ni behöver endast DirectXTex projektet.
  - Länka in som Lib fil i ert projekt (ModelViewer, Game, etc).
  - Kan länka in i er GraphicsEngine om ni vill.
    - Kom ihåg vad som händer när ni länkar lib filer i lib filer!

- Vi kommer bara att behöva en liten del av detta…
  - Men ni kommer nog på fler områden senare.

TEXTURDATA

- Med DirectXTex kan vi nu enkelt ladda en textur...

```
DirectX::TexMetadata textureMetadata{};
DirectX::ScratchImage textureImage;
HRESULT result = DirectX::LoadFromDDSFile(aNoiseTexture.c_str(), DirectX::DDS_FLAGS_NONE, &textureMetadata, [&]textureImage);
```

- Och sedan avkomprimera den till rå RGBA.

```
const DirectX::Image* compressedImage = textureImage.GetImage(mip:0, item:0, slice:0);
DirectX::ScratchImage uncompressedImage;
result = DirectX::Decompress(*compressedImage, DXGI_FORMAT_R32G32B32A32_FLOAT, [&]uncompressedImage);
```

- Nu har vi data vi kan använda!
  - Tänk bara på att pixlarna är byte-data.
  - Ni måste veta deras stride, alltså storlek.

```
const uint8_t* pixelPtr = uncompressedImage.GetPixels();
const size_t pixelPtrSize = uncompressedImage.GetPixelsSize();
const size_t pixelSize = pixelPtrSize / (textureMetadata.width * textureMetadata.height);
```

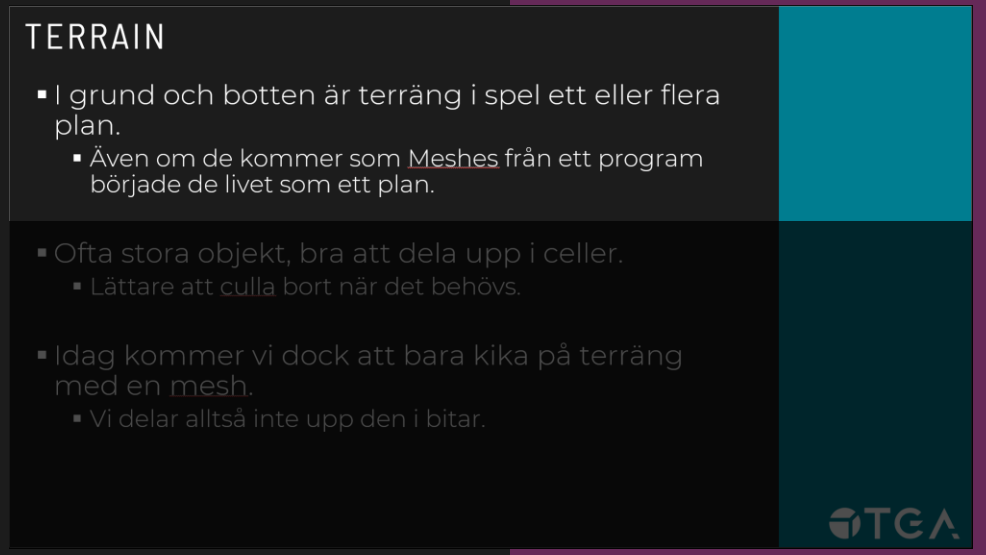
TGA

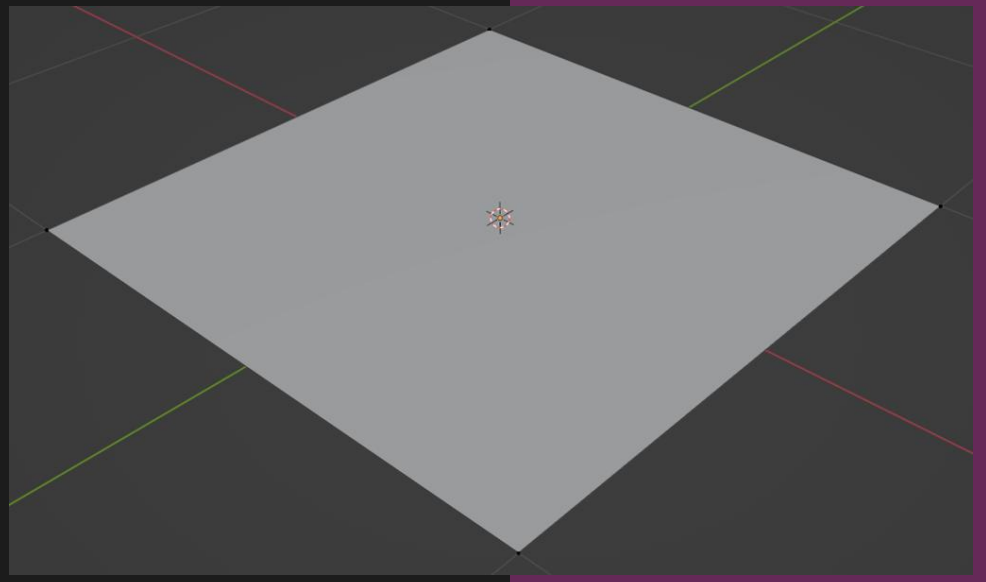
TEXTURDATA

- Men hur läser vi en jättelång `char*` med pixlar?
  - Den är endast en dimension, bilder är två.
- Vi läser den radvis.
- Om ni funderar en liten stund kan ni säkert konvertera pixel pekaren till en 2D Array.
- Bara tänk på vad ni loopar på när ni översätter.

TERRAIN

- Vi pratade om att terräng är ett plan.
  - På slide 6.
- Plan kan vi redan skapa.
  - Kuben från förra terminen är ju 6st plan.
- Det vi måste tänka på speciellt med terräng är upplösningen.
  - Alltså hur många vertices vi har.
- Planen i vår kub har bara 4st vertices.
  - Det räcker inte.

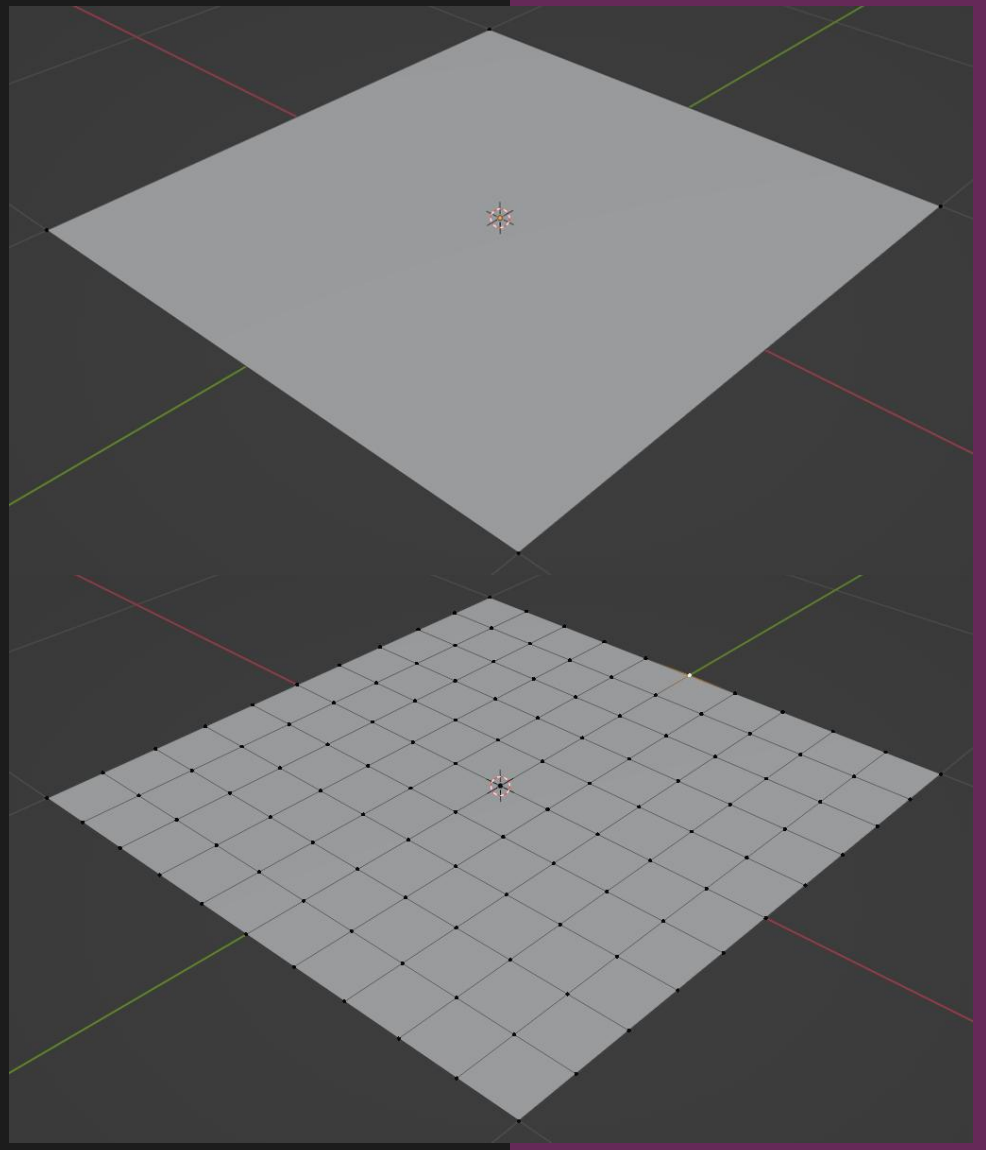




TGA

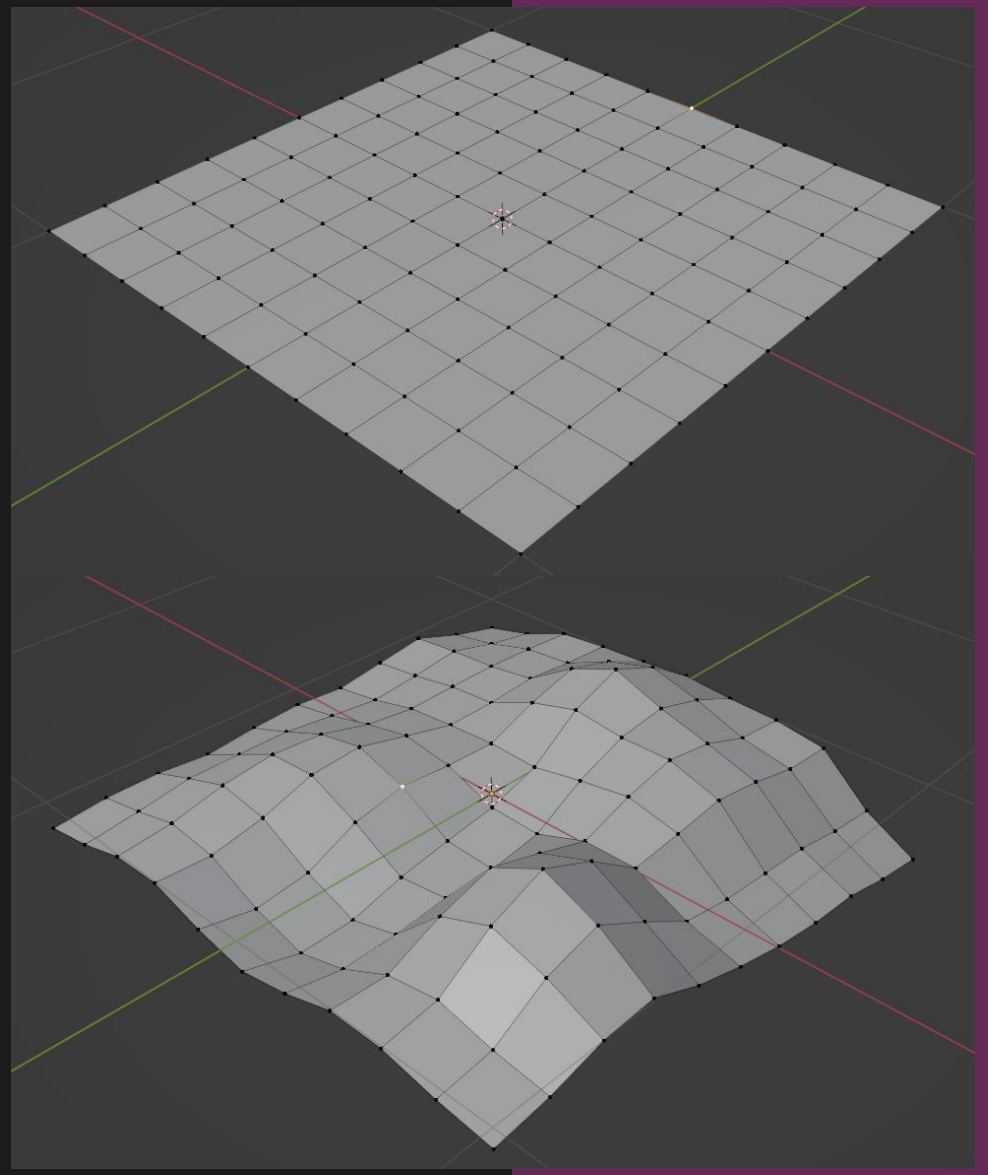
TERRAIN

- Varför inte?
- Kanske kommer ihåg att en Vertex är en Control Point.
  - Ett ställe där något händer.
- För att vårt plan ska ha en yta med detaljer behöver vi flera vertices.
- Viktigt att tänka på när ni gör terräng!



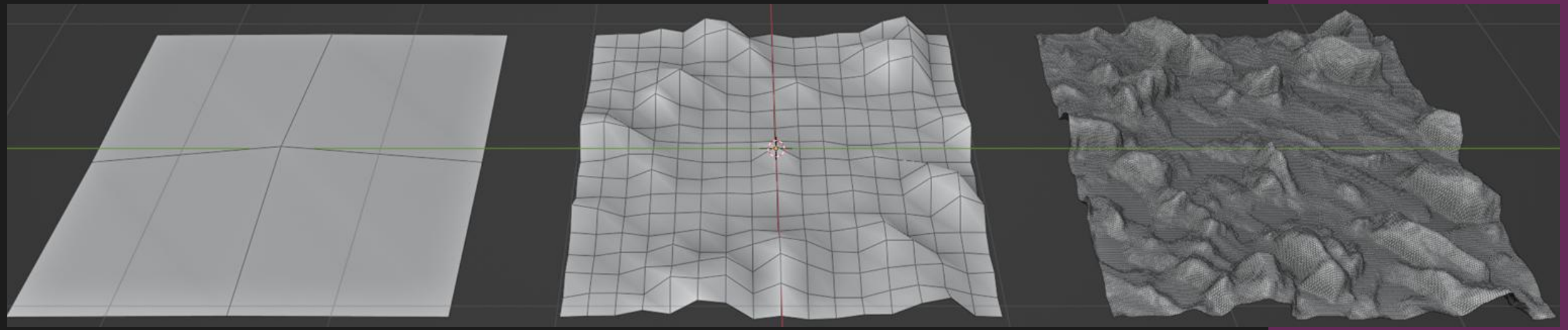
TERRAIN

- Att göra ett plan till terräng är enkelt.
  - Ett plan är ju platt. Y = 0.
- Med vårt Height Field kan vi offsetta Y positionen baserat på ett värde.
- Resultatet blir då en mesh som grafiskt representerar vårt Height Field.
  - Ett terrängplan!



TERRAIN

- Exempel på terrängplan med olika mängder vertices.
- Gissning på hur många? Planet är 2x2m.



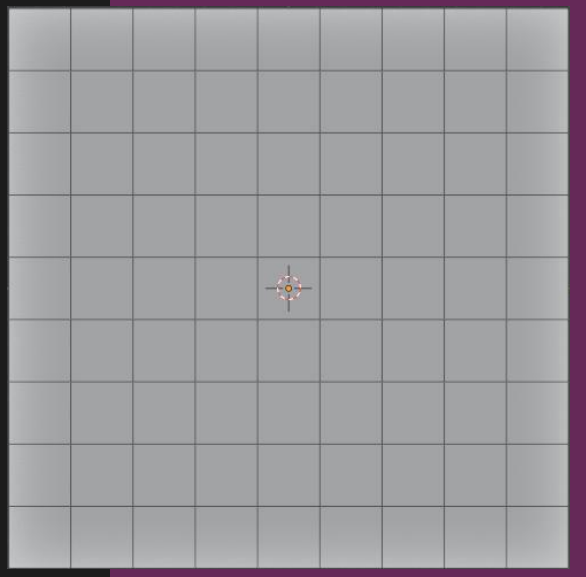
9 289 66049

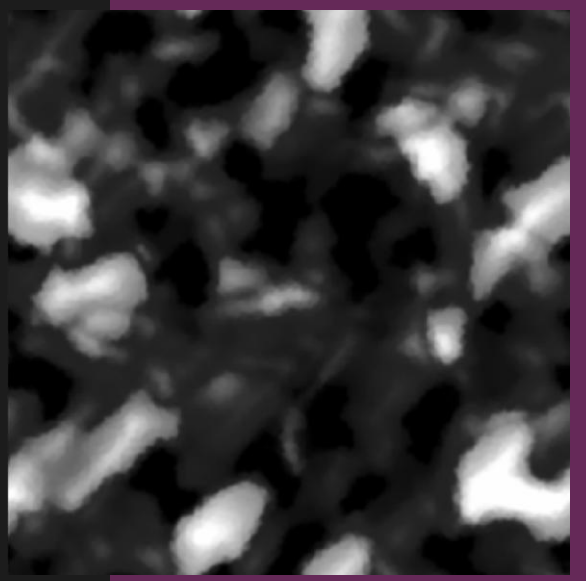
TERRAIN

- Vad behöver vi då tänka på när vi bygger terräng?

- Ni kommer nog inte ha 1:1 mellan vertices och pixlar.
  - Om er terräng inte är väldigt liten.

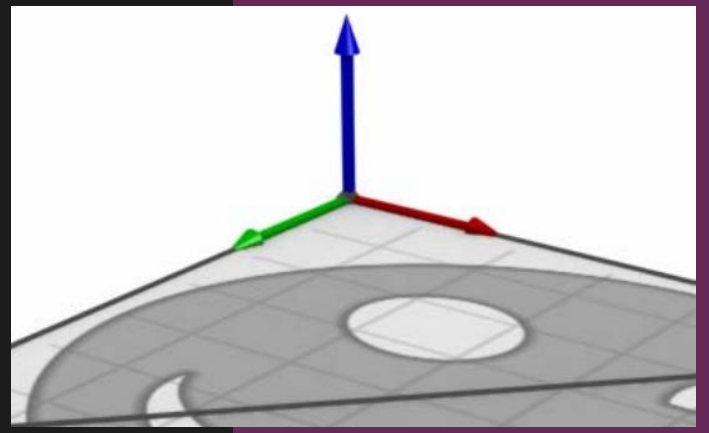
- Ni måste beräkna var i ert Height Field ni ska läsa för att få data.
  - Kan vara bra att kunna läsa ”mellan” pixlar och lerpa?





TERRAIN

- Men hur beräknar vi normaler?
  - Som ni vet finns endast geometrinormaler för Vertices.
  - Det kan vi dra nytta av.



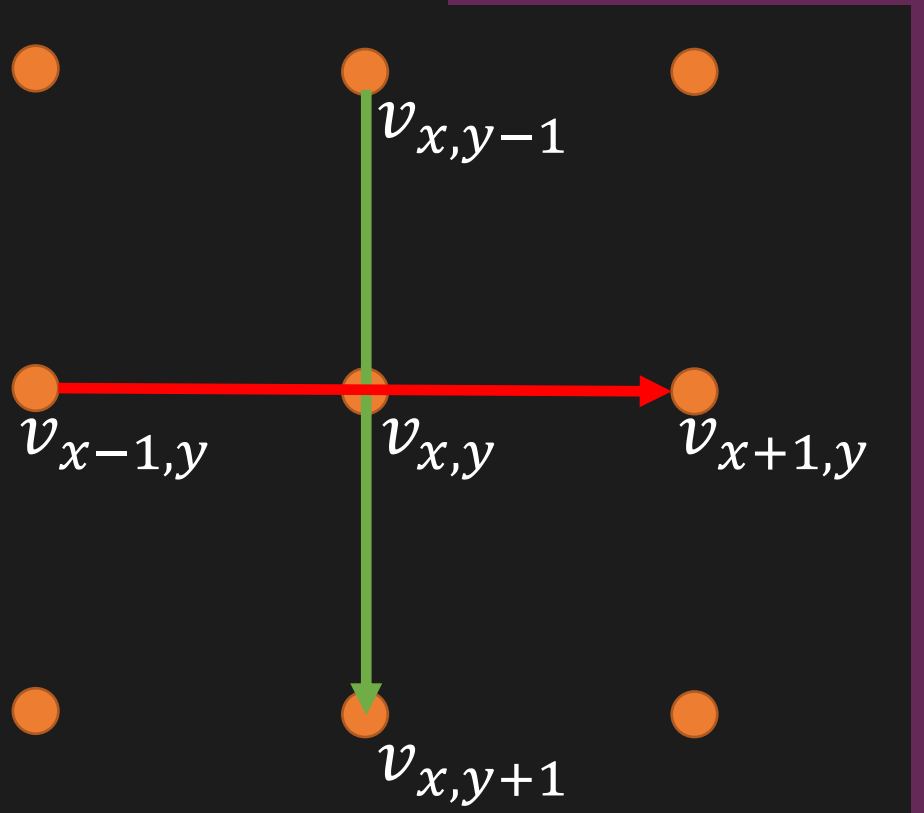
- Hur får vi en jämn övergång mellan normaler?
  - Normalens Tangent och Binormal pekar mot nästa.

- Vi kan helt enkelt beräkna normalen med kryssprodukten av dessa.
  - En Tangent Vector följer ytan vid en given punkt.

$$\vec{N} = \vec{T}_a \times \vec{T}_b$$

TERRAIN

- Beräkna Normalen för vertexen $v_{x,y}$.
  - $\vec{N}_{v_{x,y}}$
  - Ska följa ytan.
- Vi tar vår formel igen.
  - $\vec{N} = \vec{T}_a \times \vec{T}_b$
- $\vec{T}_a$ är vektorn som går uppifrån och ner.
  - $\vec{T}_a = v_{x,y+1} - v_{x,y-1}$
  - Binormalen.
- $\vec{T}_b$ är vektorn som går vänster till höger.
  - $\vec{T}_b = v_{x+1,y} - v_{x-1,y}$
  - Tangenten.

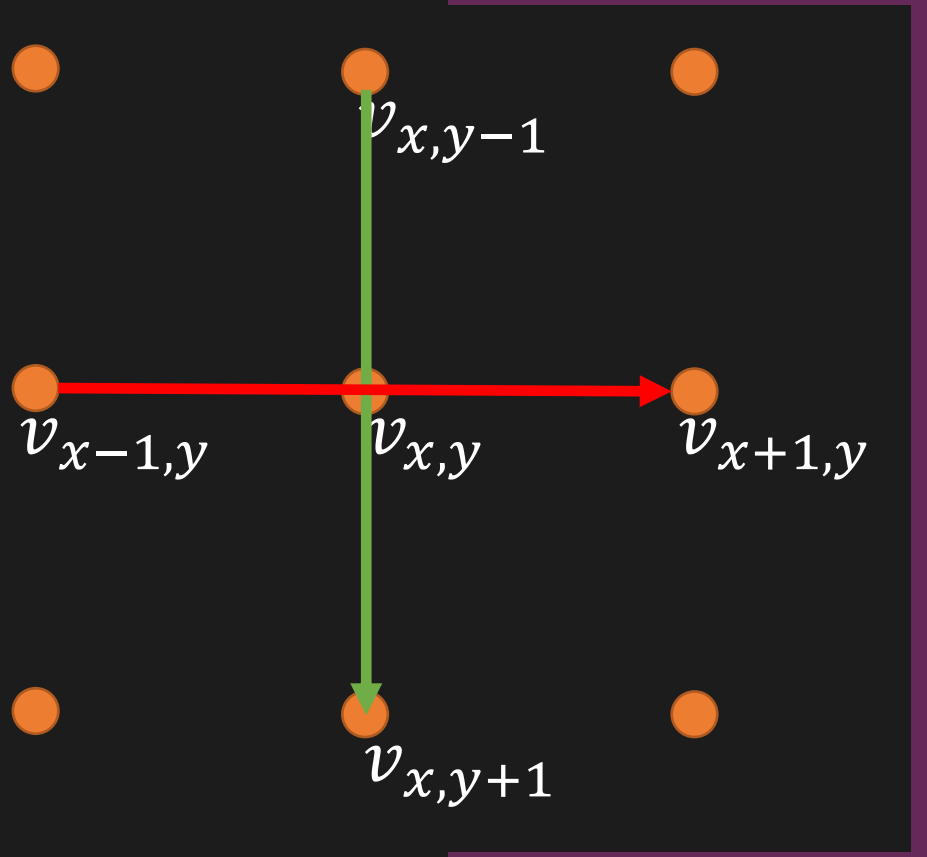


TERRAIN

$$\vec{N}_{v_{x,y}} = \vec{T}_a \times \vec{T}_b$$

eller

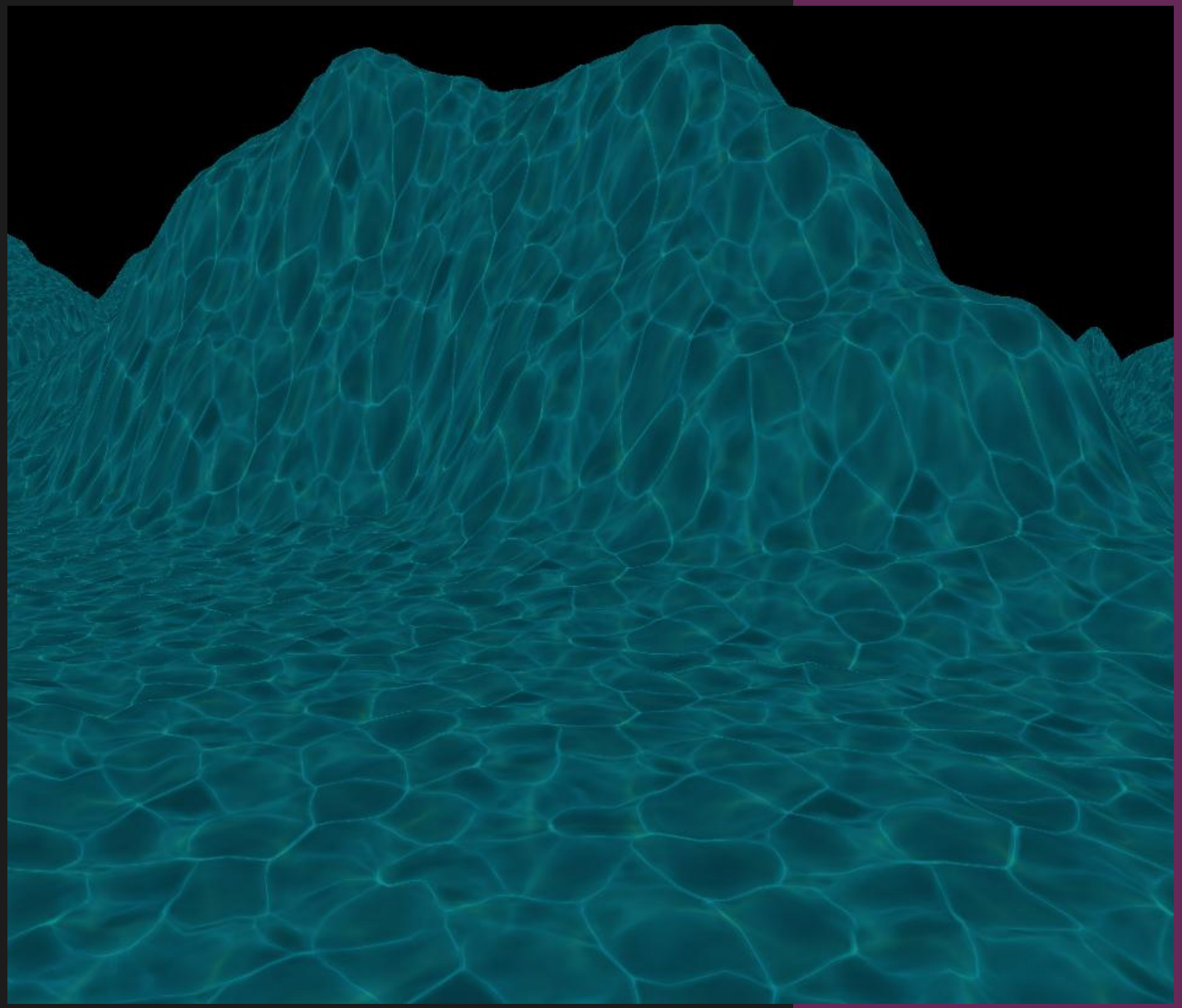
$$\vec{N}_{v_{x,y}} = (v_{x,y+1} - v_{x,y-1}) \times (v_{x+1,y} - v_{x-1,y})$$



- Om x = 3, y = 7 blir det
  - $(3, 7 + 1) - (3, 7 - 1) \times (3 + 1, 7) - (3 - 1, 7)$
- Normaler är alltid $0 \leq N \leq 1$ men ekvationen behöver inte vara det.
- Normalisera alltid era Normaler!

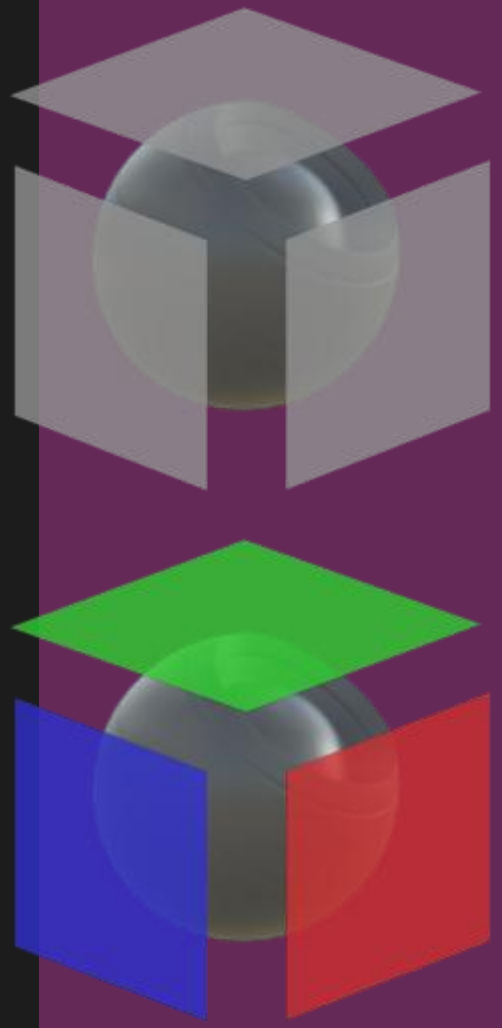
TERRAIN

- Nu har vi terräng med normaler!
  - Jättebra.
- Men slänger vi på en textur blir det väldigt fult.
- Texturen sträcks ut vid höjdskillnader.
- Varför?



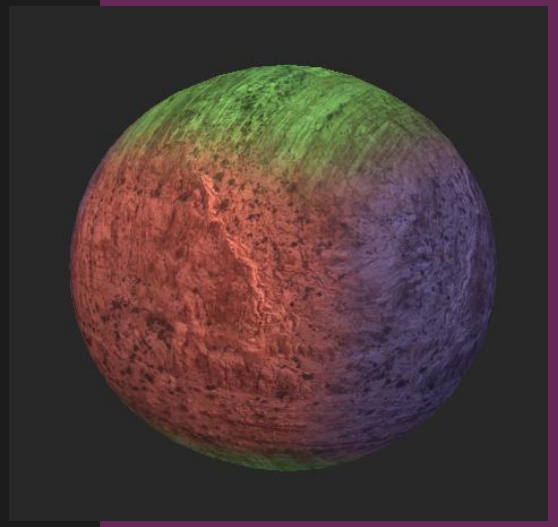
TERRAIN

- Detta är relativt lätt att fixa.
  - Vi struntar i att använda UV för att läsa på texturen!
- Vi kan istället projicera texturen på vår terräng.
  - Men vad betyder då det?
- Vi definierar tre texturer på tre oändliga plan, ett per axelriktning.
- Detta ger oss också namnet på denna tekniken.



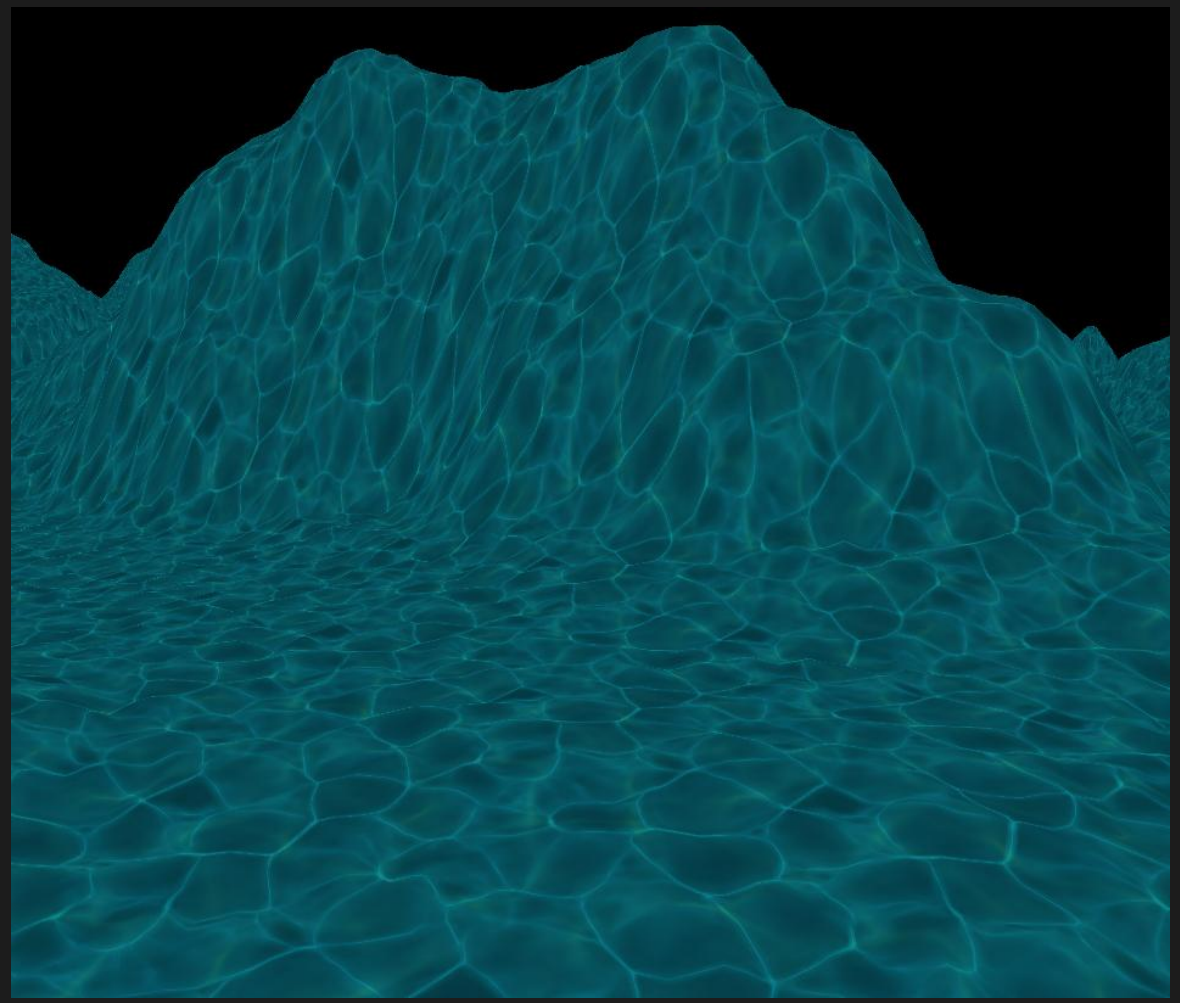
TRIPLANAR MAPPING

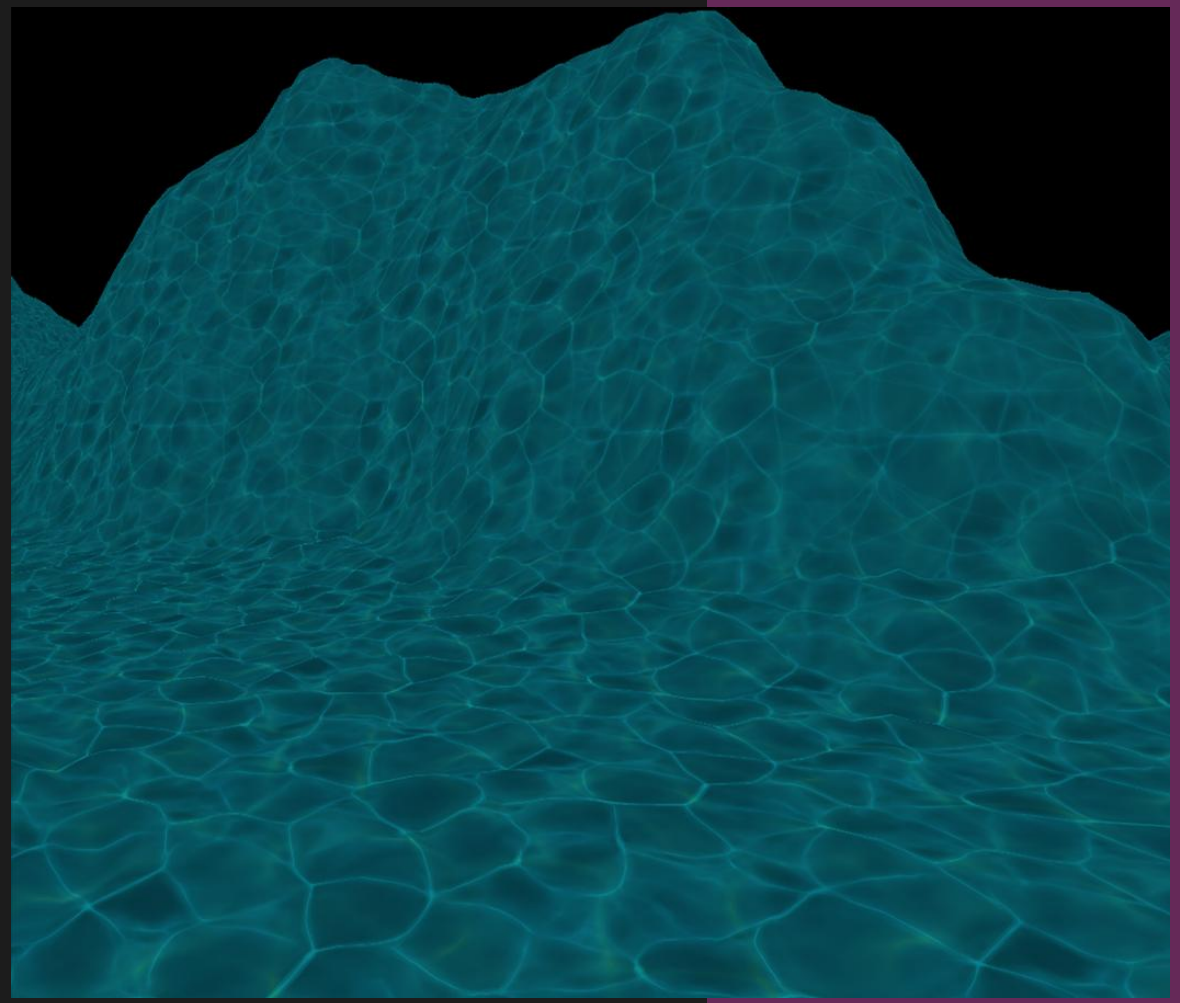
- Genom att hämta texturdata i dessa tre planen kan vi undvika sträckningar.
  - Detta eftersom vi hämtar texturen från det plan som är närmast normalriktningen.
- Med denna teknik på vår terräng ser allting mycket bättre ut.
- Vi tar upp en jämförelse…





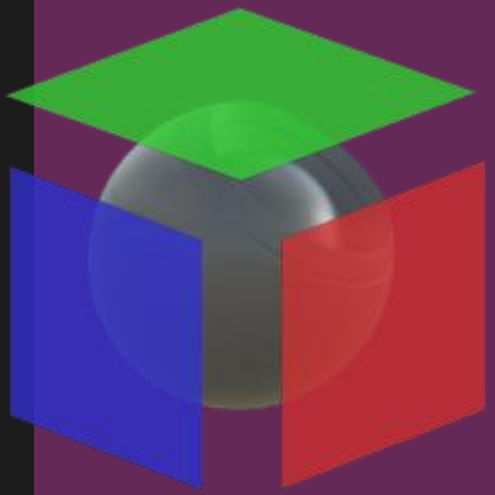
TRIPLANAR MAPPING





TRIPLANAR MAPPING

- Hur vet vi vilket plan vi vill använda?
  - Och om vi inte använder UV koordinater, vad gör vi då?



- Vi använder UV koordinater, men inte de som finns i vår Vertex.

- Istället måste vi skapa dem från de tre planen.

- Detta kan vi göra med pixelns World Position.

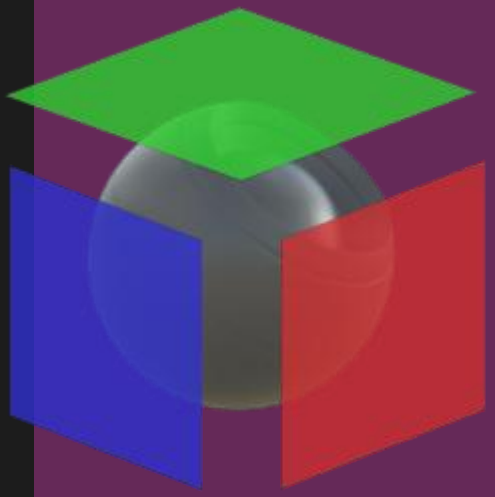
TRIPLANAR MAPPING

- Från World Position kan vi definiera 3 oändliga plan.

```
// X plane has U = Z, V = Y
const float2 uvX = aWorldPosition.zy;

// The Y plane has U = X, V = Z
const float2 uvY = aWorldPosition.xz;

// The Z plane has U = X and V = Y
const float2 uvZ = aWorldPosition.xy;
```



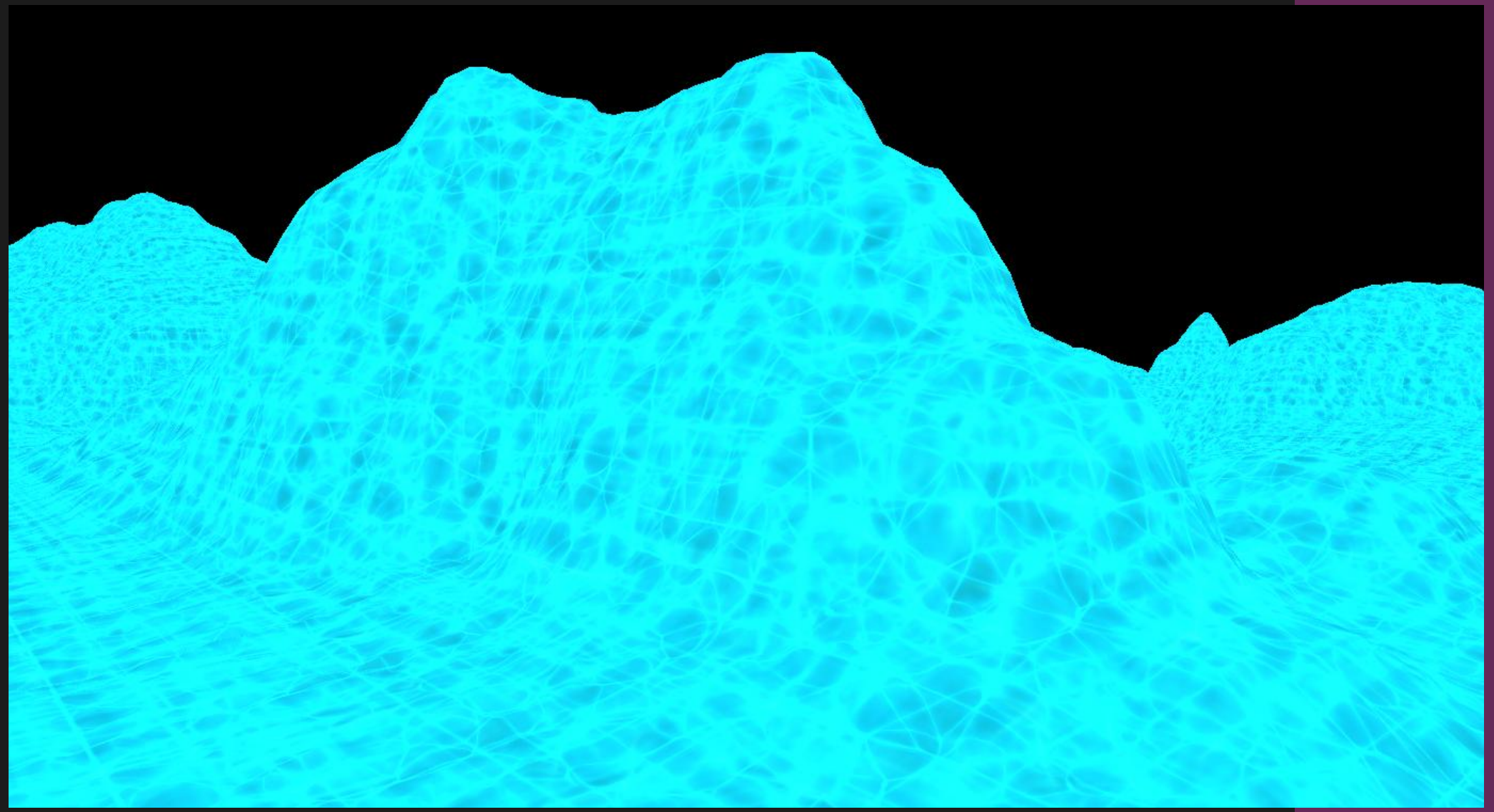
- Vi kan tekniskt sett använda dessa direkt!

```
const float4 sampleX = aTexture.Sample(wrapSampler, uvX);
const float4 sampleY = aTexture.Sample(wrapSampler, uvY);
const float4 sampleZ = aTexture.Sample(wrapSampler, uvZ);

return sampleX + sampleY + sampleZ;
```

- Men det ser inte bra ut alls. Alla pixlar använder alla texturplanen.

USCH



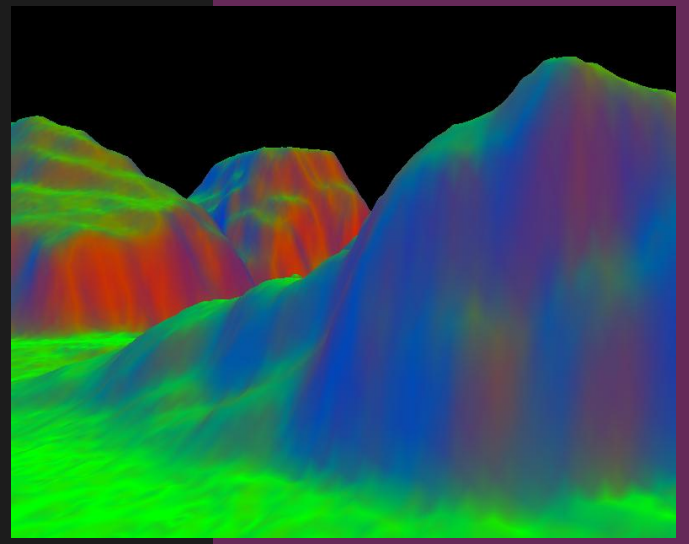
TRIPLANAR MAPPING

- Istället måste vi hitta något ställe att välja plan baserat på normalens riktning.

- Vi kan enkelt göra detta genom att nolla riktningar vår normal inte pekar åt.

```
float blend = abs(aWorldNormal);
blend /= blend.x + blend.y + blend.z;
return sampleX * blend.x + sampleY * blend.y + sampleZ * blend.z;
```

- Problemet är att vi inte sållar bort planen ordentligt.
  - Vi ser flera texturer ovanpå varandra.





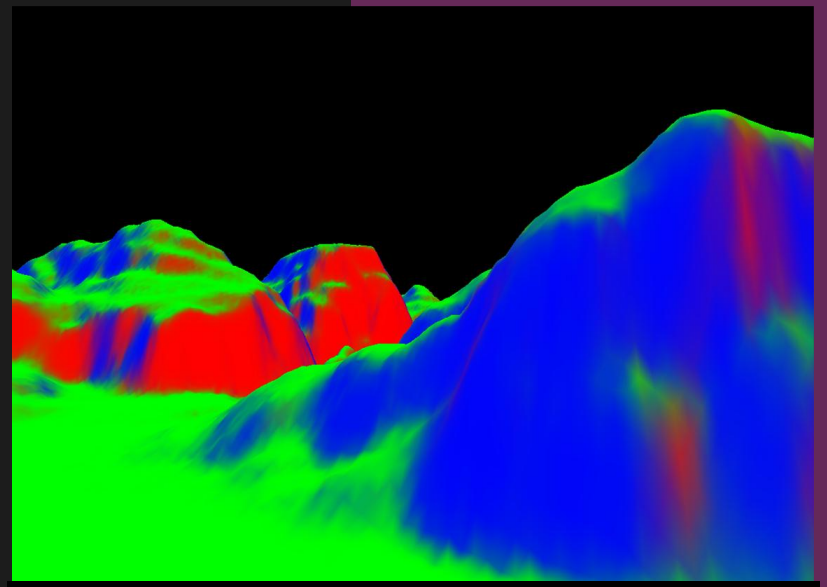
TRIPLANAR MAPPING

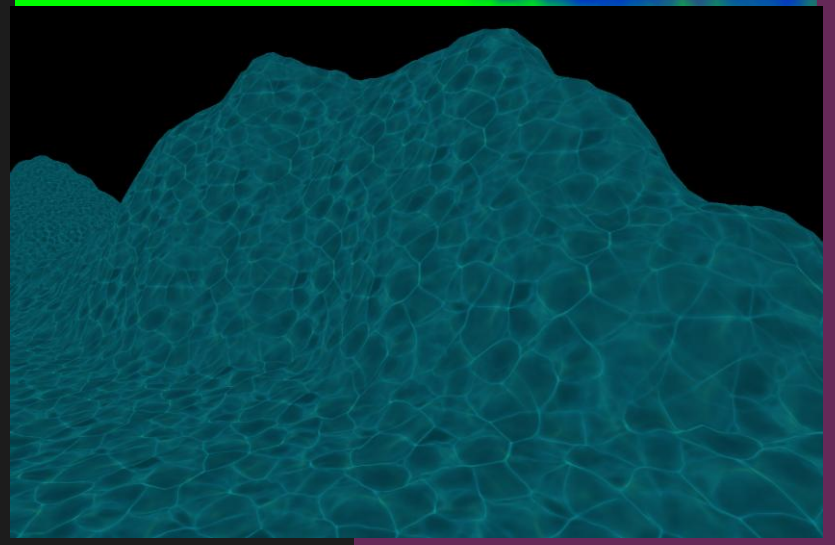
- Vi kan skärpa till övergångarna mellan planen.
  - Höj vår World Normal till en Power.

```
float3 blend = pow(aWorldNormal, 4);
blend /= dot(blend, float3(1, 1, 1));
```

- Detta ger skarpare kanter.

- Använder vi sedan dessa får vi bättre övergångar.
  - Dock är normaler med runt 45 grader fortfarande lite fel.
  - Märks inte så ofta i terräng.





TRIPLANAR MAPPING

- Nu ser det rätt bra ut men vi får väldigt konstiga fel med Normal Maps.
  - Suck.
- Anledningen är att de ofta hamnar åt fel håll och detaljer krockar!
- Vi kan fixa detta med olika typer av blending för normaler.
- Den vi ska titta på heter Whiteout Blending.

TRIPLANAR BLENDING

- Whiteout Blending kommer från SIGGRAPH 07.
  - AMDs Ruby: Whiteout demo
  - Beskrevs av Christopher Oat

- Blendar normaler korrekt per plan utan att skala om axlarna.
  - Ska fortfarande normaliseras!

```
float3 sampleX = ExpandNormal(aNormalTexture.Sample(wrapSampler, uvX).rgb);
float3 sampleY = ExpandNormal(aNormalTexture.Sample(wrapSampler, uvY).rgb);
float3 sampleZ = ExpandNormal(aNormalTexture.Sample(wrapSampler, uvZ).rgb);

// Swizzle into Tangent and use Whiteout Blending
sampleX = float3(sampleX.xy + aWorldNormal.zy, abs(sampleX.z) * aWorldNormal.x);
sampleY = float3(sampleY.xy + aWorldNormal.xz, abs(sampleY.z) * aWorldNormal.y);
sampleZ = float3(sampleZ.xy + aWorldNormal.xy, abs(sampleZ.z) * aWorldNormal.z);

return normalize(sampleX.zyx * blend.x + sampleY.xzy * blend.y + sampleZ.xyz * blend.z);
```

FLERA MATERIAL

- Som ni sett på min terräng har jag flera olika material.
  - Vatten, Sand, Gräs, Sten och Snö.
- Hur blandar vi dessa?
  - Finns flera metoder, som vanligt.
  - Alla kräver flera texturer.
  - Jag har en del.
  - Tänk på slottarna!
- För blending använder jag Vertex Colors.
  - Men ni kan lika gärna lägga till Height till er Vertex.

```
Texture2D baseAlbedoTexture : register(t0);
Texture2D baseNormalTexture : register(t1);
Texture2D baseMaterialTexture : register(t2);

Texture2D layer1AlbedoTexture : register(t3);
Texture2D layer1NormalTexture : register(t4);
Texture2D layer1MaterialTexture : register(t5);

Texture2D layer2AlbedoTexture : register(t6);
Texture2D layer2NormalTexture : register(t7);
Texture2D layer2MaterialTexture : register(t8);

Texture2D layer3AlbedoTexture : register(t9);
Texture2D layer3NormalTexture : register(t10);
Texture2D layer3MaterialTexture : register(t11);

Texture2D layer4AlbedoTexture : register(t12);
Texture2D layer4NormalTexture : register(t13);
Texture2D layer4MaterialTexture : register(t14);
```

FLERA MATERIAL

- Efter att ha samplat på alla material lägger jag ihop dem.
  - Resultatet blir det material eller övergång som är rätt.

```
MaterialLayerData layerResult;
layerResult.Albedo =
    Layer1.Albedo * input.myVxColor.r +
    Layer2.Albedo * input.myVxColor.g +
    Layer3.Albedo * input.myVxColor.b +
    Layer4.Albedo * input.myVxColor.a;
layerResult.Normal =
    Layer1.Normal * input.myVxColor.r +
    Layer2.Normal * input.myVxColor.g +
    Layer3.Normal * input.myVxColor.b +
    Layer4.Normal * input.myVxColor.a;
layerResult.Material =
    Layer1.Material * input.myVxColor.r +
    Layer2.Material * input.myVxColor.g +
    Layer3.Material * input.myVxColor.b +
    Layer4.Material * input.myVxColor.a;
```

- Sen gör jag en mask som är svart när det inte finns VX color och lerpar in baslagret.

```
const float baseLayerMask = saturate(input.myVxColor.r + input.myVxColor.g + input.myVxColor.b + input.myVxColor.a);

const float3 landscapePixelNormal = lerp(BaseLayer.Normal, layerResult.Normal, baseLayerMask);
const float3 landscapeAlbedo = lerp(BaseLayer.Albedo, layerResult.Albedo, baseLayerMask).rgb;
const float4 landscapeMaterial = lerp(BaseLayer.Material, layerResult.Material, baseLayerMask);
```

- Sen ljussätter jag som vanligt.

FLERA MATERIAL

- Hur styr jag mina Vertex Colors?

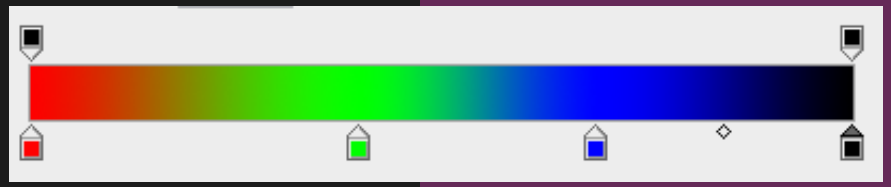
- Har gjort en Gradient klass som fungerar som Photoshops Gradient editor.

- Bör ni kunna gissa er till ganska enkelt, det är bara en lista med färger :).
  - Om position 0.0 är röd och position 0.3 är grön är positionen 0.15 gul.

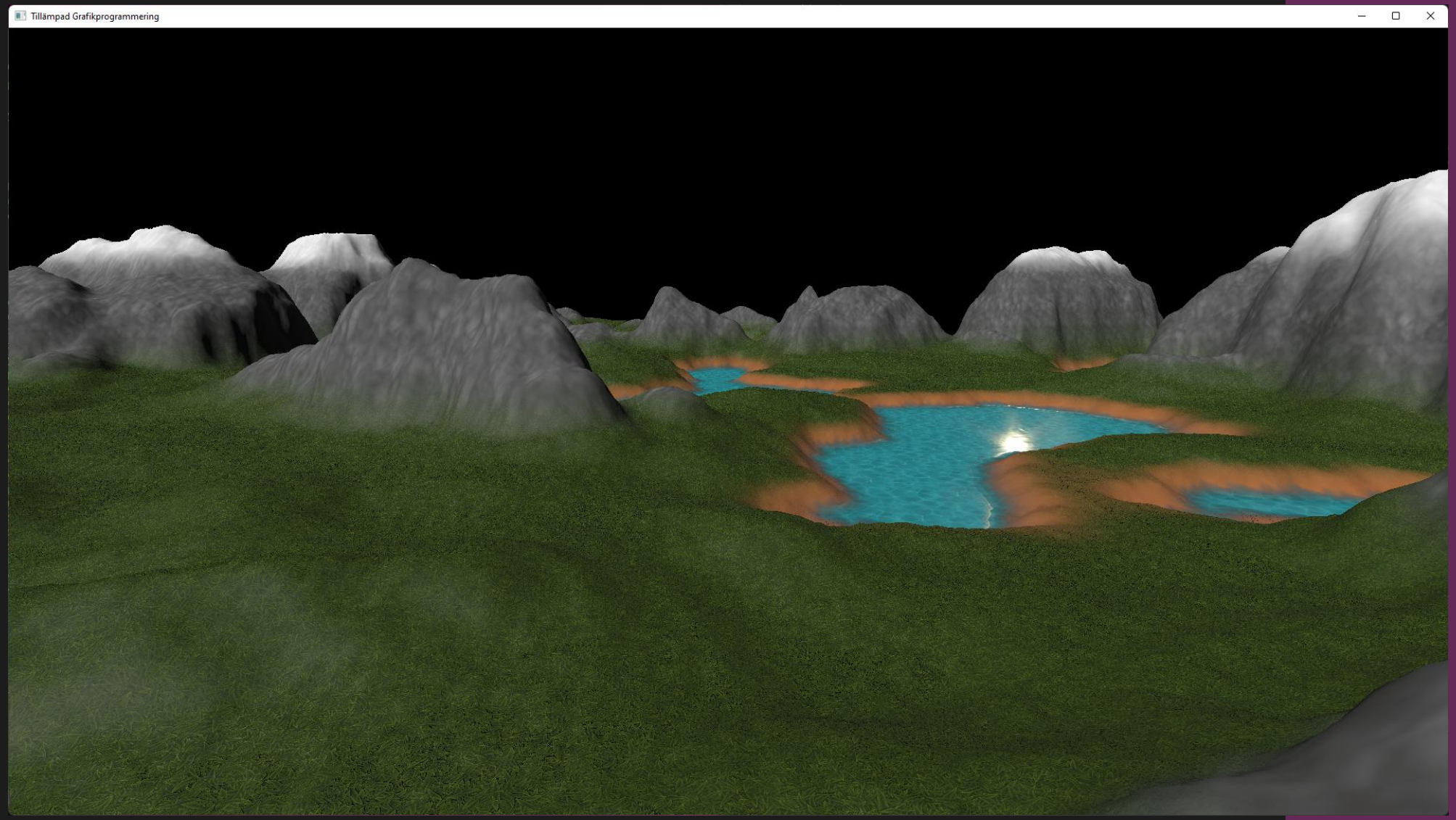
```
class Gradient
{
    std::vector<unsigned int> myKeyPositions;
    std::unordered_map<unsigned int, Vector4f> myKeys;

public:
    bool AddKey(float aValue, Vector4f aColor) { ... }

    Vector4f GetColor(float aValue) const { ... }
};
```



TERRAIN



CAVEATS

- Ni bör skriva separat pixelshader för Landscapes.
- Dyrt att köra! Många samples. Svårt att optimera.
- För production, dela upp ert landscape i bitar.
  - Finns flera olika sätt för detta. Googla!
  - Låter er bara rendera delar av landskapet som syns.
- Läs gärna mer om noise :)
  - Scape: 2. Procedural basics
  - Scape: 3. Procedural extensions

FRÅGOR?

